

Agenti logici: calcolo proposizionale

Claudio Gallicchio

a.a. 2023/2024

Riferimenti



AIMA Cap 7

■ 7.5, 7.6

Dimostrazione di teoremi

Dimostrazione di teoremi nella logica proposizionale

- Fin qui abbiamo visto come determinare la **conseguenza logica tramite il model checking**
 - enumerare i modelli e mostrare che la formula deve valere in tutti
- La conseguenza logica puo' essere ottenuta anche tramite la **dimostrazione di teoremi**
 - **applicando regole di inferenza** direttamente alle formule della KB
 - costruire una dimostrazione della formula desiderata **senza consultare i modelli**

Dimostrazione di teoremi nella logica proposizionale

Tre fondamentali concetti relativi alla
conseguenza logica:

1. Equivalenza logica
2. Validità
3. Soddisfacibilità



Equivalenza logica

- Due formule α e β sono equivalenti se sono vere nello stesso insieme di modelli
- $\alpha \equiv \beta$ se e solo se $\alpha \models \beta$ e $\beta \models \alpha$

Esempi:

$$A \wedge B \equiv B \wedge A$$

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

$$(A \wedge (A \vee B)) \equiv A$$



Il ruolo di queste equivalenze è analogo a quello delle identità aritmetiche in matematica.

Equivalenze logiche (leggi)

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$$

$$\neg(\neg\alpha) \equiv \alpha$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$$

commutatività di $\wedge$

commutatività di $\vee$

associatività di $\wedge$

associatività di $\vee$

eliminazione della doppia negazione

contrapposizione

eliminazione dell'implicazione

eliminazione del bicondizionale

De Morgan

De Morgan

distributività di $\wedge$ su $\vee$

distributività di $\vee$ su $\wedge$





Validità

- Una formula α **valida** sse è vera in tutte le interpretazioni
 - Es. $A \vee \neg A$
- Le formule valide sono anche dette **tautologie**
 - sono formule necessariamente vere
- **Teorema di deduzione**
 - date due formule α e β : $\alpha \models \beta$ se e solo se $(\alpha \Rightarrow \beta)$ è valida





Soddisfacibilità

- Una formula α è **soddisfacibile** sse esiste una interpretazione in cui α è vera (ovvero se esiste un **modello** di α)
 - SAT: determinare la soddisfacibilità di formule della logica proposizionale
- Validità e soddisfacibilità sono strettamente connesse
 - α è **valida** sse $\neg\alpha$ è **insoddisfacibile**
 - α è **soddisfacibile** sse $\neg\alpha$ non è **valida**

Dimostrazione per assurdo

- Th. di deduzione: $\alpha \models \beta$ se e solo se $(\alpha \Rightarrow \beta)$ è valida
 - $\alpha \models \beta$ se e solo se $\neg(\alpha \Rightarrow \beta)$ è insoddisfacibile
 - $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$ [eliminazione dell'implicazione]
 - $\neg(\alpha \Rightarrow \beta) \equiv \neg(\neg\alpha \vee \beta) \equiv (\alpha \wedge \neg\beta)$ [De Morgan]
 - $\alpha \models \beta$ se e solo se $(\alpha \wedge \neg\beta)$ è insoddisfacibile
 - dimostrazione per refutazione o per contraddizione
 - si assume che β sia falsa e si dimostra che questo porta a una contraddizione con gli assiomi α
-

Inferenza per PROP

■ *Model checking*

- una forma di inferenza che fa riferimento alla definizione di conseguenza logica (si enumerano i possibili modelli)
- Tecnica delle tabelle di verità (TV)

■ Algoritmi per la *soddisfacibilità* (SAT)

- $KB \models \alpha$ sse $(KB \wedge \neg \alpha)$ è insoddisfacibile (*Teorema di refutazione*)
 - ✓ dimostrare α partendo da KB verificando che $(KB \wedge \neg \alpha)$ non è mai vero (dimostrazione per assurdo)
 - ✓ si parte da $\neg \alpha$ e si dimostra che questo porta a una contraddizione con gli assiomi in KB (che sono noti)
- La conseguenza logica può essere ricondotta a un problema SAT

Model checking

Model checking

Consiste in:

- Enumerare tutti i possibili modelli di KB
 - ✓ tramite il calcolo della tavola di verità (TV)
- Verificare che in essi la formula α sia vera

$$M(KB) \subseteq M(\alpha)$$



L'algoritmo TV-Consegue?

- Quesito: α è conseguenza logica di KB? ($KB \models \alpha$?)
- Enumera tutte le possibili interpretazioni di KB
 - (k simboli, 2^k possibili interpretazioni)
- Per ciascuna interpretazione
 - Se non soddisfa KB, OK
 - Se soddisfa KB, si controlla che soddisfi anche α
- Se si trova anche solo una interpretazione che soddisfa KB e non α la risposta sarà NO.

Funzione TV-Consegue?

```
function TV-Consegue?( $KB, \alpha$ ) // returns true oppure false  
    inputs:  $KB$ , la base di conoscenza, una formula della logica proposizionale  
             $\alpha$ , la query, una formula della logica proposizionale  
  
     $simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$   
    return TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )
```

```
function TV-Verifica-Tutto( $KB, \alpha, simboli, modello$ ) // returns true oppure false
```

```
if Vuoto?( $simboli$ ) then
```

```
    if PL-Vero?( $KB, modello$ ) then return PL-Vero?( $\alpha, modello$ )
```

```
    else return true // quando  $KB$  è false, restituisce sempre true
```

```
else do
```

```
     $P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )
```

```
    return TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )
```

```
        and
```

```
        TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )
```

PL-Vero($KB, model$)
restituisce True se KB è
vera nella
interpretazione *model*.

$modello$ rappresenta un
modello parziale (in cui
sono stati assegnati
valori solo ad alcune
variabili)

Tabella di verità

$$\overbrace{(\neg a \vee b) \wedge (a \vee c)}^{\text{KB}} \models \overbrace{(b \vee c)}^{\text{formula}}$$



a	b	c	$\neg a \vee b$	$a \vee c$	$b \vee c$
T	T	T	T	T	T
T	T	F	T	T	T
F	T	T	T	T	T
F	F	T	T	T	T

individuo le
interpretazioni che
sono modelli di KB

verifico se la
formula è vera in
tutti i modelli

Esempio di TV-Consegue?

KB: $(\neg a \vee b) \wedge (a \vee c)$

α : $(b \vee c)$

simboli = $[a, b, c]$

- TV-VERIFICA-TUTTO(KB, α , $[a, b, c]$, $\{ \}$)
 - TV-VERIFICA-TUTTO(KB, α , $[b, c]$, $\{a=T\}$)
 - ✓ TV-VERIFICA-TUTTO(KB, α , $[c]$, $\{a=T, b=T\}$)
 - TV-VERIFICA-TUTTO(KB, α , $[]$, $\{a=T, b=T, c=T\}$) OK
 - TV-VERIFICA-TUTTO(KB, α , $[]$, $\{a=T, b=T, c=F\}$) OK
 - ✓ TV-VERIFICA-TUTTO(KB, α , $[c]$, $\{a=T, b=F\}$)
 - TV-VERIFICA-TUTTO(KB, α , $[]$, $\{a=T, b=F, c=T\}$) OK
 - TV-VERIFICA-TUTTO(KB, α , $[]$, $\{a=T, b=F, c=F\}$) OK
 - TV-VERIFICA-TUTTO(KB, α , $[b, c]$, $\{a=F\}$) ...

In questi due casi
l'interpretazione è un modello
di KB e di α
 $PL\text{-Vero?}(KB, \text{modello}) = \text{true}$
 $PL\text{-Vero?}(\alpha, \text{modello}) = \text{true}$

In questi due casi
l'interpretazione non è un
modello di KB
 $PL\text{-Vero?}(KB, \text{modello}) = \text{false}$

Solo alla fine, dopo avere provato tutti i possibili assegnamenti (interpretazioni) possiamo rispondere se $(b \vee c)$ è o meno conseguenza logica.



Algoritmi per la soddisfacibilità (SAT)

- Ne vediamo alcuni che usano KB in **forma a clausole**
- La forma a clausole è la **forma normale congiuntiva (CNF)**: una congiunzione di disgiunzioni di letterali

$$(A \vee B) \wedge (\neg B \vee C \vee D) \wedge (\neg A \vee F)$$

- Non è restrittiva: è sempre possibile ottenerla con trasformazioni che preservano l'equivalenza logica
- Può essere rappresentata come insiemi di letterali

$$\{A, B\} \{\neg B, C, D\} \{\neg A, F\}$$

Clausole

- Una clausola è una disgiunzione di letterali
- Un letterale è dato
 - da un simbolo
 - dalla negazione di un simbolo

Es.

$$A \vee B \vee \neg C \vee D$$

$$\neg A \vee B \vee C \vee \neg D$$

Trasformazione in forma a clausole

I passi sono:

1. Eliminazione del $\Leftrightarrow$: $(A \Leftrightarrow B) \equiv (A \Rightarrow B) \wedge (B \Rightarrow A)$
2. Eliminazione dell' $\Rightarrow$: $(A \Rightarrow B) \equiv (\neg A \vee B)$
3. Negazioni all'interno:
 $\neg(A \vee B) \equiv (\neg A \wedge \neg B)$
 $\neg(A \wedge B) \equiv (\neg A \vee \neg B)$
(De Morgan)
4. Distribuzione di $\vee$ su $\wedge$:
 $(A \vee (B \wedge C)) \equiv (A \vee B) \wedge (A \vee C)$

Esempio di trasformazione

C'è brezza in [1,1] sse c'è un pozzo in [1,2] o [2,1]

1. $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

2. $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$

3. $(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1})) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$

4. $(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1})) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$

5. $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

6. $\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\}$



L'algoritmo DPLL per la soddisfacibilità

- **DPLL**: Davis, Putman, e poi Logemann, Loveland
- Parte da una KB **in forma a clausole**
 - prende in input una formula in CNF (cioè, un insieme di clausole)
- È una enumerazione ricorsiva *in profondità* di tutte le possibili interpretazioni alla ricerca di un modello
- Tre miglioramenti rispetto a TV-Consegue?:
 1. Terminazione anticipata
 2. Euristica dei simboli (o letterali) puri
 3. Euristica delle clausole unitarie

DPLL: terminazione anticipata

- Si può decidere sulla verità di una clausola anche con interpretazioni parziali: basta che un letterale sia vero
 - Se A è vero, lo sono anche $\{A, B\}$ e $\{A, C\}$ indipendentemente dai valori di B e C
- Se anche una sola clausola è falsa l'interpretazione non può essere un modello dell'insieme di clausole
 - Dato $\{C_1\}\{C_2\}$, se C_1 è falso lo è anche $\{C_1\}\{C_2\}$

(cioè $C_1 \wedge C_2$)

DPLL: simboli puri

- *Simbolo puro*: un simbolo che appare con lo stesso «segno» in tutte le clausole
Es. $\{A, \neg B\} \{\neg B, \neg C\} \{C, A\}$ A è puro, B anche
- Nel determinare se un simbolo è puro se ne possono trascurare le occorrenze in clausole già rese vere
- I simboli puri possono essere assegnati a *True* se il letterale è positivo, *False* se negativo.
- Non si eliminano modelli utili: se le clausole hanno un modello continuano ad averlo dopo questo assegnamento (non renderà mai falsa una clausola in cui compare il simbolo puro)

DPLL: clausole unitarie

- *Clausola unitaria*: una clausola con un solo letterale *non assegnato*

Es. $\{B\}$ è unitaria ma anche ...

$\{B, \neg C\}$ è unitaria quando $C = \text{True}$

- Conviene assegnare prima valori al letterale in clausole unitarie. L'assegnamento è obbligato (*True* se positivo, *False* se negativo).

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return** *true*

if qualche clausola in *clausole* è falsa in *modello* **then return** *false*

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

 DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

DPLL prende un insieme di clausole, un insieme di simboli da assegnare (inizialmente tutti i simboli presenti nelle clausole), e un assegnamento di verità ai simboli, inizialmente vuoto.

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return** *true*

if qualche clausola in *clausole* è falsa in *modello* **then return** *false*

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

 DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return true**

if qualche clausola in *clausole* è falsa in *modello* **then return false**

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

Si controlla che tutte le clausole siano soddisfatte dall'assegnamento, anche parziale, in tal caso si restituisce vero.

L'algoritmo DPLL

```
function DPLL-Soddisfacibile?(s) returns true oppure false  
    inputs: s, una formula della logica proposizionale  
    clausole  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di s  
    simboli  $\leftarrow$  una lista di tutti i simboli proposizionali in s  
    return DPLL(clausole, simboli, { })
```

```
function DPLL(clausole, simboli, modello) returns true oppure false  
    if ogni clausola in clausole è vera in modello then return true  
    if qualche clausola in clausole è falsa in modello then return false  
    P, valore  $\leftarrow$  Trova-Simbolo-Puro(simboli, clausole, modello)  
    if P è diverso da null then return DPLL(clausole, simboli – P, modello  $\leftarrow$  {P = valore})  
    P, valore  $\leftarrow$  Trova-Clausola-Unitaria(clausole, modello)  
    if P è diverso da null then return DPLL(clausole, simboli – P, modello  $\leftarrow$  {P = valore})  
    P  $\leftarrow$  Primo(simboli); resto  $\leftarrow$  Resto(simboli)  
    return DPLL(clausole, resto, modello  $\leftarrow$  {P = true}) or  
           DPLL(clausole, resto, modello  $\leftarrow$  {P = false})
```

Si controlla se (anche solo) una delle clausole non è soddisfatta, in tal caso falso.

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return** *true*

if qualche clausola in *clausole* è falsa in *modello* **then return** *false*

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

se esiste un simbolo puro, lo restituisce insieme al suo valore (True, se compare positivo, False, se compare negato)

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return true**

if qualche clausola in *clausole* è falsa in *modello* **then return false**

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

Se esiste viene richiamato DPLL dopo avere esteso il modello con l'assegnamento al simbolo puro e aver tolto il simbolo dai simboli ancora da assegnare.

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return** *true*

if qualche clausola in *clausole* è falsa in *modello* **then return** *false*

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

se esiste una clausola unitaria restituisce il simbolo della clausola unitaria e il suo valore (True, se compare positivo, False, se compare negato)

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return true**

if qualche clausola in *clausole* è falsa in *modello* **then return false**

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

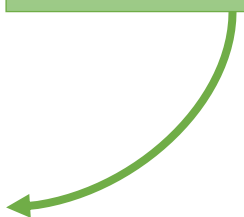
if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

Se esiste viene richiamato DPLL dopo avere esteso il modello con l'assegnamento al simbolo puro e aver tolto il simbolo dai simboli ancora da assegnare.



L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return true**

if qualche clausola in *clausole* è falsa in *modello* **then return false**

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})

Altrimenti si prende il primo simbolo da assegnare e ricorsivamente si richiama DPLL provando ad assegnare Vero o Falso. Se uno dei due ha successo si termina con successo.

DPLL: esempio

KB $\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \models \{\neg P_{1,2}\} ?$

Aggiungiamo $\{P_{1,2}\}$ e vediamo se l'insieme è insoddisfacibile

SAT($\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \{P_{1,2}\}) ?$

1

2

3

4

5

1. La 5 è unitaria; $P_{1,2} = \text{True}$; la prima clausola e la 5 sono soddisfatte

$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \{P_{1,2}\}$

2. $P_{2,1}$ è un simbolo puro; $P_{2,1} = \text{False}$; 3 è soddisfatta

$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \{P_{1,2}\}$

...

Non esistono modelli quindi $\neg P_{1,2}$ è conseguenza logica della KB



Miglioramenti di DPLL

- DPLL è completo e termina sempre
- Alcuni miglioramenti:
 - Analisi di componenti (sotto-problemi indipendenti): se le variabili possono essere suddivise in sotto-insiemi disgiunti (senza simboli in comune)
 - ✓ un risolutore può lavorare separatamente su ciascun componente
 - Ordinamento di variabili e valori: scegliere la variabile che compare in più clausole
 - Backtracking intelligente e altre ottimizzazioni ...

Metodi locali per SAT: formulazione

- Gli 'stati' sono interpretazioni, assegnamenti **completi**
- L'obiettivo è un assegnamento che soddisfa tutte le clausole (un modello)
- Si parte da un assegnamento casuale
- Ad ogni passo si cambia il valore di un simbolo proposizionale (*flip*)
- Gli stati sono valutati contando il numero di clausole **soddisfatte** (più sono meglio è)

Metodi locali per SAT: algoritmi

- Ci sono molti massimi locali per sfuggire ai quali serve introdurre perturbazioni casuali
 - Hill climbing con riavvio casuale
 - Simulated Annealing
- Molta sperimentazione per trovare il miglior compromesso tra il grado di “avidità” e casualità
- **WALK-SAT** è uno degli algoritmi più semplici ed efficaci



WalkSAT

- WalkSAT ad ogni passo
 - Sceglie a caso una clausola non ancora soddisfatta
 - Individua un simbolo da modificare (*flip*), scegliendo con probabilità p (di solito 0.5) tra:
 - ✓ Scegliere un simbolo a caso (**passo casuale**, *random walk*)
 - ✓ Scegliere quello che rende più clausole soddisfatte (**passo di ottimizzazione**)
- Si arrende dopo un certo numero di *flip* predefinito

WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```


WalkSAT: un esempio

Come euristica usiamo il numero di clausole soddisfatte (da massimizzare)

Rosso: passo casuale

Verde: passo di ottimizzazione

$$\begin{array}{cccc} \{\neg B_{1,1}, P_{1,2}, P_{2,1}\} & \{\neg P_{1,2}, B_{1,1}\} & \{\neg P_{2,1}, B_{1,1}\} & \{\neg B_{1,1}\} \\ 1 & 2 & 3 & 4 \end{array}$$

[$B_{1,1}=F, P_{1,2}=T, P_{2,1}=T$] 2, 3 F; scelgo 2; a caso flip $B_{1,1}$

[$B_{1,1}=T, P_{1,2}=T, P_{2,1}=T$] 4 F; scelgo 4; flip $B_{1,1}$ unica scelta

[$B_{1,1}=F, P_{1,2}=T, P_{2,1}=T$] 2, 3 F; scelgo 2; a caso flip $P_{1,2}$

[$B_{1,1}=F, P_{1,2}=F, P_{2,1}=T$] 3 F; scelgo 3; ottimizzazione: flip $P_{2,1}$ [#4]; flip $B_{1,1}$ [#3]

[$B_{1,1}=F, P_{1,2}=F, P_{2,1}=F$] ho trovato un modello!!!

Analisi di WalkSAT

- Se *max-flips* = ∞ e l'insieme di clausole è soddisfacibile prima o poi termina.
- Ma se vogliamo un algoritmo che termina, questo è necessariamente incompleto
- Va bene per cercare un modello, sapendo che c'è, ma se è insoddisfacibile non termina
- Non può essere usato per verificare l'insoddisfacibilità

Problemi SAT difficili

*cioè con un numero relativamente
piccolo di clausole*

- Se un problema ha molte soluzioni (problema sotto-vincolato) è più probabile che WalkSAT ne trovi una in tempi brevi
- Istanza di 3-SAT (clausole con tre letterali)
 $\{\neg D, \neg B, C\} \{B, \neg A, \neg C\} \{\neg C, \neg B, E\} \{E, \neg D, B\} \{B, E, \neg C\}$
 - Esempio: 16 soluzioni su 32; un assegnamento ha il 50% di probabilità di essere giusto: 2 passi in media!

Problemi SAT difficili

- Se un problema ha molte soluzioni (problema sotto-vincolato) è più probabile che WalkSAT ne trovi una in tempi brevi
- Istanza di 3-SAT (clausole con tre letterali)

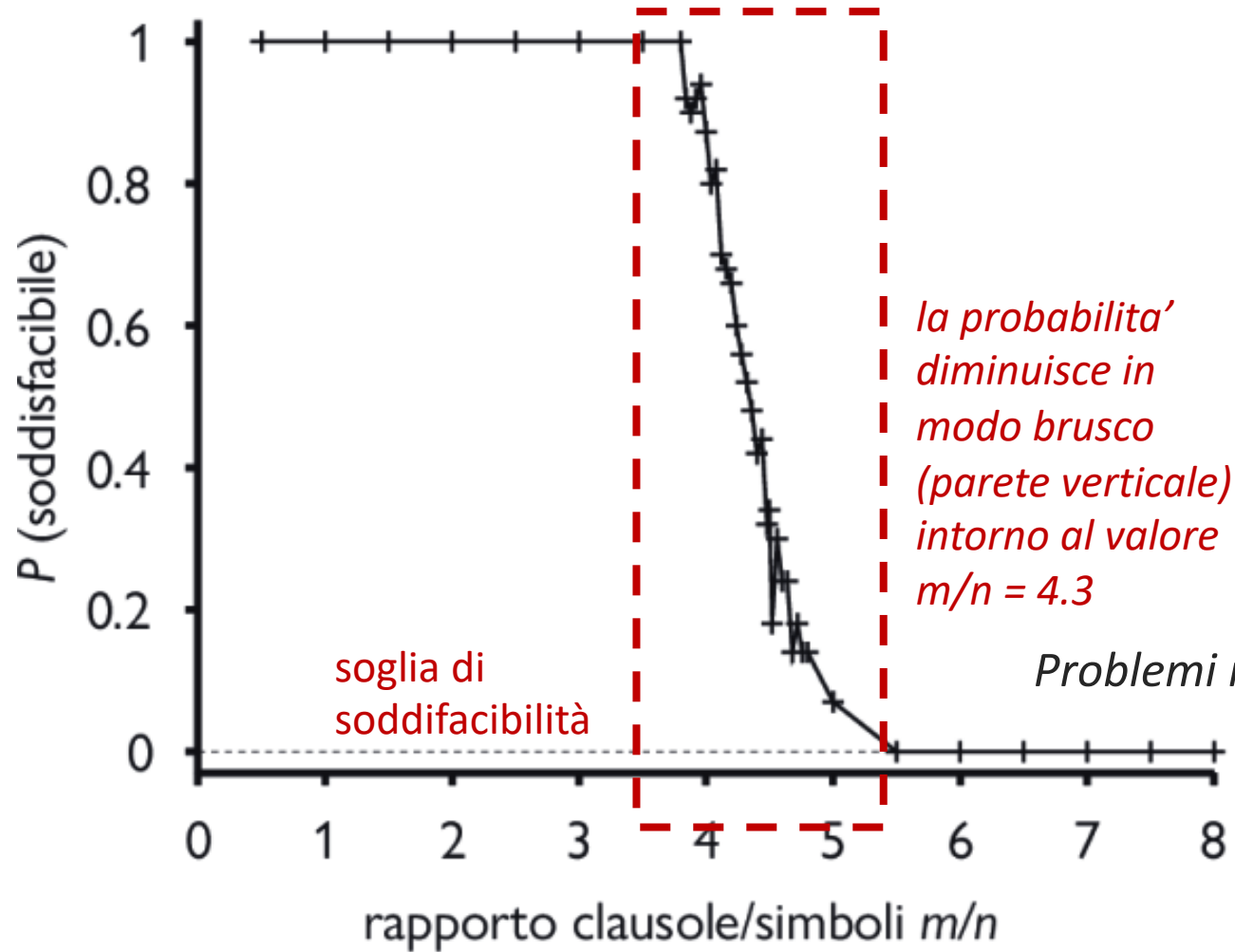
$\{\neg D, \neg B, C\} \{B, \neg A, \neg C\} \{\neg C, \neg B, E\} \{E, \neg D, B\} \{B, E, \neg C\}$

- Esempio: 16 soluzioni su 32; un assegnamento ha il 50% di probabilità di essere giusto: 2 passi in media!

- È significativo il rapporto m/n
 - m è il numero di clausole (vincoli)
 - n il numero di simboli
 - Nell'esempio: $5/5=1$
- Più grande il rapporto, più vincolato è il problema
 - Le regine sono 'facili' perché il problema è sotto-vincolato

Probabilità di soddisfacibilità in funzione di m/n

Problemi poco vincolati



m (n. clausole) varia

n (n. simboli) = 50

3 letterali per clausola

media su 100

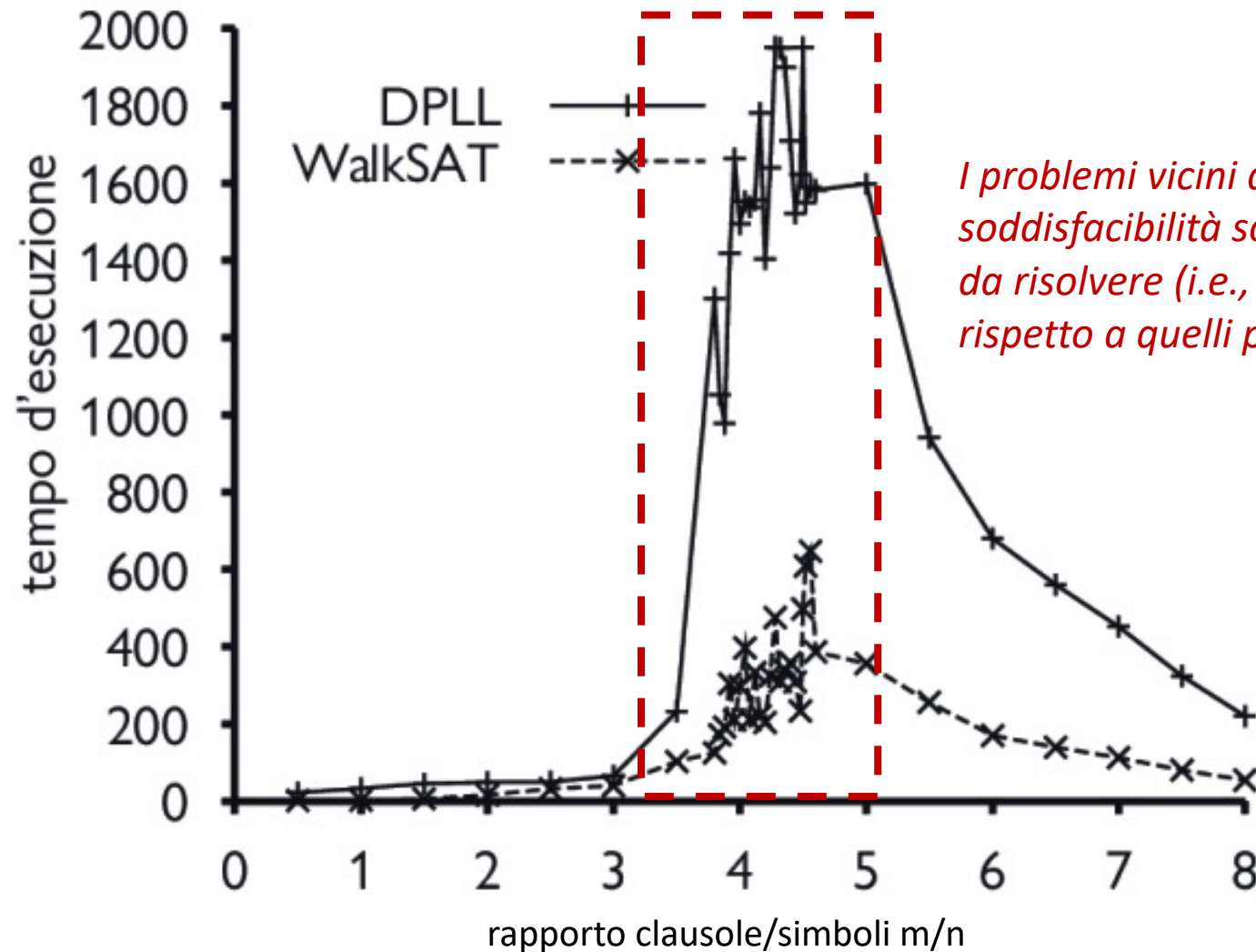
problemi generati a caso

*la probabilità
diminuisce in
modo brusco
(parete verticale)
intorno al valore
 $m/n = 4.3$*

Problemi molto vincolati

Confronto tra DPLL e WalkSAT

tempo medio di
esecuzione (misurato in
numero di iterazioni)
per DPLL e WalkSAT su
formule casuali 3-CNF





Inferenza come deduzione

- Un altro modo per decidere se $KB \models A$ è usare un sistema di deduzione
 - Denotiamo con $KB \vdash A$ il fatto che A è derivabile (deducibile) da KB
 - La deduzione avviene specificando delle regole di inferenza.
- In un sistema di inferenza le regole ...
 - dovrebbero derivare solo formule che sono conseguenza logica
 - dovrebbero derivare tutte le formule che sono conseguenza logica

Correttezza e completezza

- **Correttezza:** Se $KB \vdash \alpha$ allora $KB \models \alpha$

Tutto ciò che è derivabile è conseguenza logica. Le regole preservano la verità

- **Completezza:** Se $KB \models \alpha$ allora $KB \vdash \alpha$

Tutto ciò che è conseguenza logica è ottenibile tramite il sistema deduttivo.

Alcune regole di inferenza per Prop

Le regole sono schemi deduttivi del tipo:

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

Modus ponens

$$\frac{\alpha \wedge \beta}{\alpha}$$

Eliminazione dell'AND

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

*Eliminazione e introduzione
della doppia implicazione*

$$\frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

Una rappresentazione per il Wumpus

$$R_1: \neg P_{1,1}$$

non ci sono pozzi in [1, 1]

C'è brezza nelle caselle adiacenti ai pozzi:

$$R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

Percezioni:

$$R_4: \neg B_{1,1}$$

non c'è brezza in [1, 1]

$$R_5: B_{2,1}$$

c'è brezza in [2, 1]

$$KB = \{R_1, R_2, R_3, R_4, R_5\}$$

$$KB \models \neg P_{1,2}?$$

Dimostrazione

$$R_6: (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) \quad (R_2, \Leftrightarrow E)$$

$$R_7: (P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1} \quad (R_6, \wedge E)$$

$$R_8: \neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}) \quad (R_7, \text{contrapposizione})$$

$$R_9: \neg(P_{1,2} \vee P_{2,1}) \quad (R_4 \text{ e } R_8, \text{Modus Ponens})$$

$$R_{10}: \neg P_{1,2} \wedge \neg P_{2,1} \quad (R_9, \text{De Morgan})$$

$$R_{11}: \neg P_{1,2} \quad (R_{10}, \wedge E)$$

Dimostrazione come ricerca

Problema: come decidere ad ogni passo qual è la regola di inferenza da applicare? ... e a quali premesse? Come evitare l'esplosione combinatoria?

- Problema di esplorazione di uno spazio di stati
- Una *procedura di dimostrazione* definisce:
 - la direzione della ricerca
 - la strategia di ricerca

Direzione della ricerca

- Nella dimostrazione di teoremi conviene procedere all'indietro.
- Con una applicazione *in avanti* delle regole di inferenza non controllata:

Da $A, B: A \wedge B$ $A \wedge (A \wedge B)$... $A \wedge (A \wedge (A \wedge B))$

- Meglio *all'indietro*
 - se si vuole dimostrare $A \wedge B$, si cerchi di dimostrare A e poi B
 - se si vuole dimostrare $A \Rightarrow B$, si assuma A e si cerchi di dimostrare B
 - ...

Strategia di ricerca

■ Completezza

- Le regole della deduzione naturale sono un insieme di regole di inferenza completo (2 per ogni connettivo)
- Se l'algoritmo di ricerca è completo siamo a posto

■ Efficienza

- La complessità è alta: è un problema decidibile ma NP-completo



Regola di risoluzione per PROLOG

- Meno regole sono meglio è, senza rinunciare alla completezza.
- Un'unica regola: la **regola di risoluzione** (presuppone forma a clausole)

$$\frac{\{P, Q\} \quad \{\neg P, R\}}{\{Q, R\}}$$

$$\frac{(P \vee Q) \wedge (\neg P \vee R)}{(Q \vee R)}$$

- Corretta? Basta pensare ai modelli
- Preferita la notazione insiemistica: gli eventuali duplicati si eliminano

La regola di risoluzione in generale

$$\frac{\{l_1, l_2, \dots, l_i, \dots, l_k\} \quad \{m_1, m_2, \dots, m_j, \dots, m_n\}}{\{l_1, l_2, \dots, l_{i-1}, l_{i+1}, \dots, l_k, m_1, m_2, \dots, m_{j-1}, m_{j+1}, \dots, m_n\}}$$

Gli l e m sono **letterali**, simboli di proposizione positivi o negativi;
 l_i e m_j sono uguali e di segno opposto

Caso particolare

$$\frac{\{P\} \quad \{\neg P\}}{\{\}}$$

*clausola vuota
contraddizione*

Il grafo di risoluzione

c'è brezza in [1,1,] sse c'è un pozzo in una delle caselle adiacenti

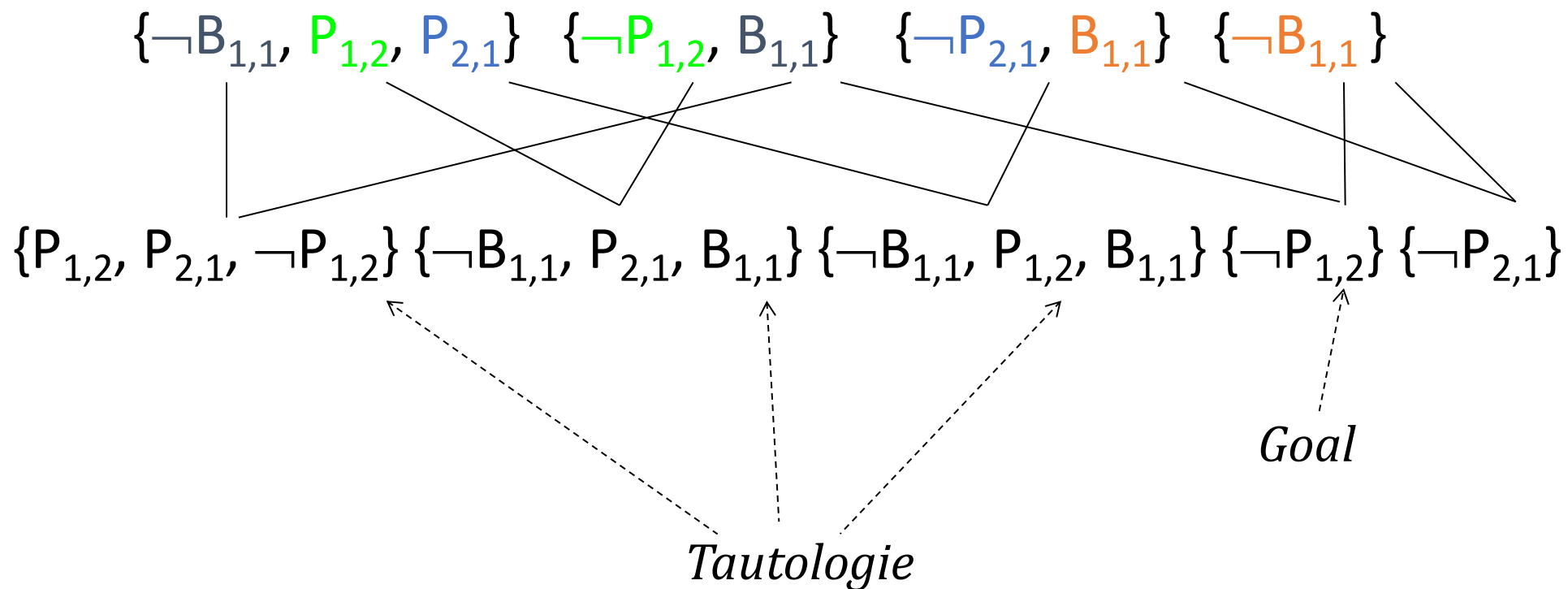
$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\}$ $\{\neg P_{1,2}, B_{1,1}\}$ $\{\neg P_{2,1}, B_{1,1}\}$ $\{\neg B_{1,1}\}$

percezione

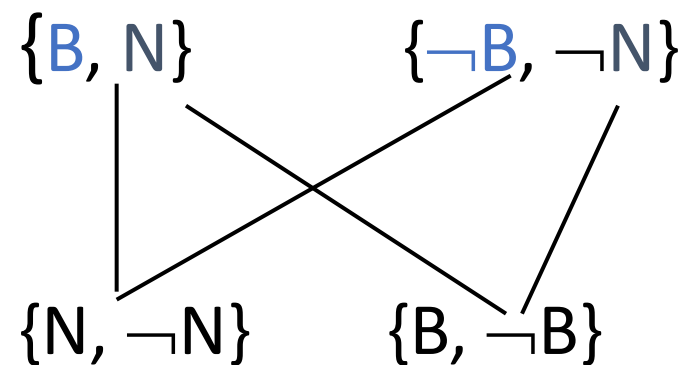
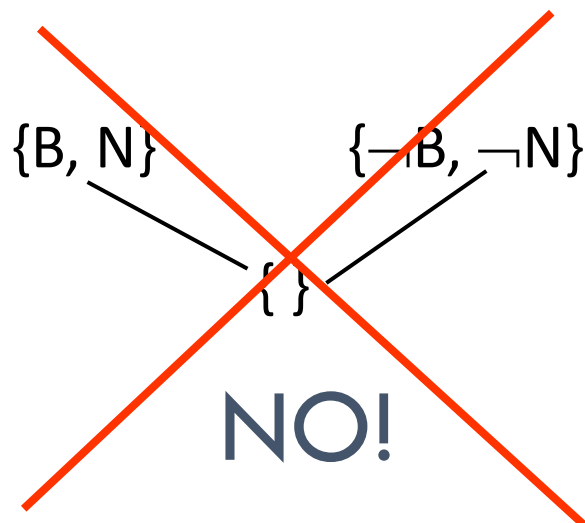
Il grafo di risoluzione

c'è brezza in [1,1,] sse c'è un pozzo in una delle caselle adiacenti

percezione



Attenzione!



... e qui ci fermiamo

Un passo alla volta !!!

Ma siamo sicuri che basti una regola? È completo?

Completezza: se $KB \models \alpha$ allora $KB \vdash_{\text{res}} \alpha$?

Non sempre. Ci basta un contro-esempio.

Es. $KB \models \{A, \neg A\}$ ma non è vero che $KB \vdash_{\text{res}} \{A, \neg A\}$

Ma siamo sicuri che basti una regola? È completo?

- Teorema di risoluzione [ground]:

KB insoddisfacibile sse $KB \vdash_{\text{res}} \{ \}$

- Teorema di refutazione offre un modo alternativo:

$KB \models \alpha$ sse $(KB \cup \{\neg\alpha\})$ è insoddisfacibile

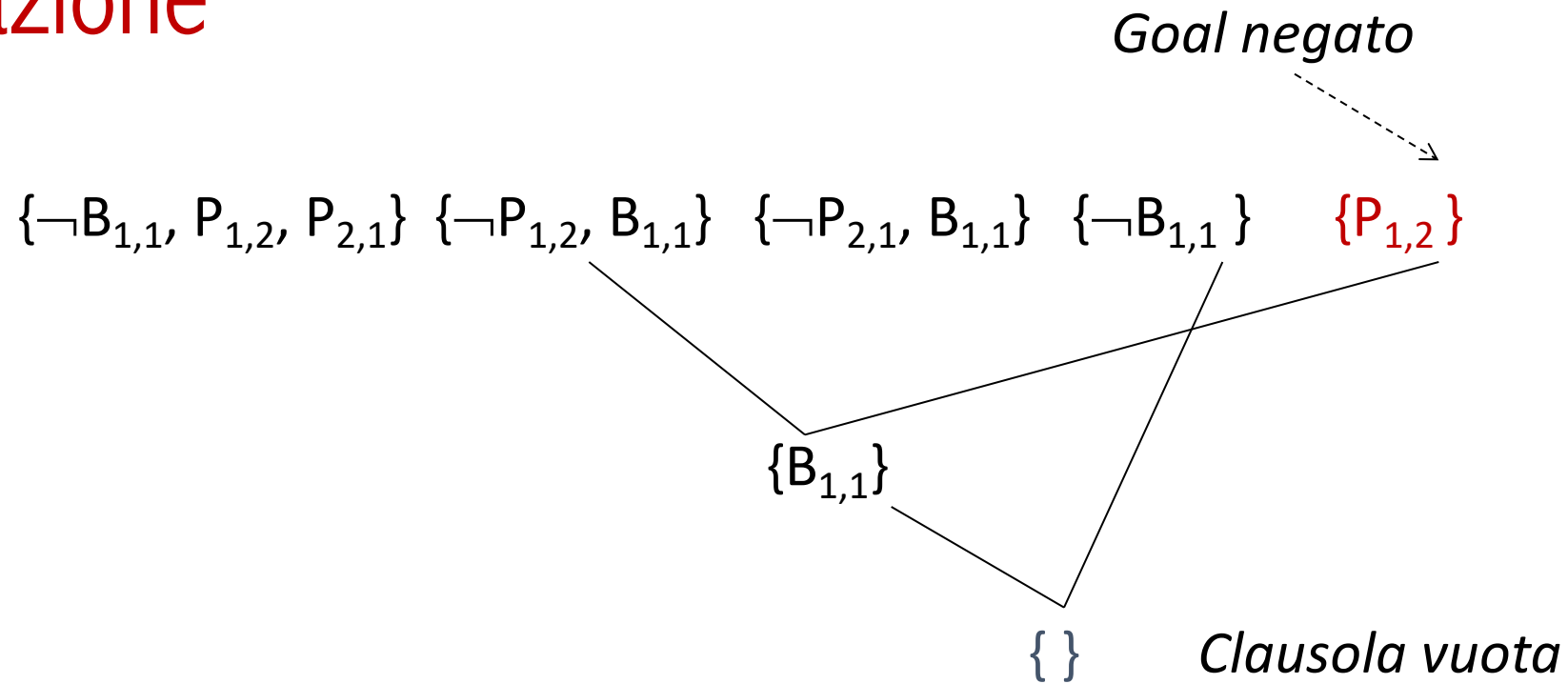
- Nell'esempio:

$KB \cup FC(\neg(A \vee \neg A))$ è insoddisfacibile? Sì, perché ...

$KB \cup \{A\} \cup \{\neg A\} \vdash_{\text{res}} \{ \}$ in un passo

Quindi possiamo concludere che $KB \models \{A, \neg A\}$

Refutazione



Conclusioni

- Abbiamo visto come gli agenti KB che usano PROP come linguaggio di rappresentazione possono decidere se $KB \models \alpha$
- Il problema è decidibile, ma intrattabile (NP) nel caso peggiore
- Esistono algoritmi efficienti e completi che consentono di affrontare problemi di grosse dimensioni
- I metodi locali sono particolarmente efficienti ma non completi